

Q-NarwhalKnight Mainnet

Block Production & Reward System

DAG-BFT Consensus + Time-Based Halving

Q-NarwhalKnight Development Team

<https://quillon.xyz>

November 11, 2025

Abstract

This document provides comprehensive technical specifications for the Q-NarwhalKnight mainnet, detailing the dual-layer architecture: high-throughput DAG-BFT consensus producing 2-10 blocks per second, and a time-based reward halving system that occurs every 4 years (calendar-based, not block-based). The system implements Bitcoin-inspired economics with an initial reward of 0.05 QUG per block, halving every 4 years until reaching a maximum supply of 21 million QUG asymptotically over approximately 256 years.

Contents

1 Executive Summary

1.1 Quick Facts

Parameter	Value
<i>DAG-BFT Consensus Layer</i>	
Block Production Rate	2-10 blocks/second
Consensus Type	DAG-Knight + Narwhal
Byzantine Fault Tolerance	Asynchronous BFT
<i>Mining Reward System</i>	
Initial Block Reward	0.05 QUG
Halving Method	TIME-BASED (Calendar)
Halving Interval	Every 4 years
Development Fee	1% (0.005 QUG initially)
Miner Reward	99% (49.005 QUG initially)
<i>Economic Parameters</i>	
Maximum Supply	21,000,000 QUG
Genesis Timestamp	Oct 26, 2025 00:00 UTC
Decimals	8 (like Bitcoin)
Terminal Inflation	< 0.01% after 64 halvings

Table 1: Q-NarwhalKnight mainnet specifications

2 Dual-Layer Architecture

2.1 Layer 1: DAG-BFT Consensus

The Q-NarwhalKnight blockchain implements a high-throughput DAG-BFT consensus layer capable of producing **2-10 blocks per second**:

- **DAG-Knight**: Zero-message complexity asynchronous BFT ordering
- **Narwhal Mempool**: High-throughput transaction batching
- **Quantum Enhancements**: VDF-based anchor election
- **Anonymous Mesh**: Tor-integrated validator network

Key Insight: Unlike traditional blockchains with fixed block times, the DAG-BFT layer produces blocks *continuously* as validators reach consensus on transaction batches.

2.2 Layer 2: Mining Reward System

The reward distribution layer operates *independently* from block production:

- **Time-Based Halving**: Rewards halve every 4 calendar years
- **Fixed Per-Block Reward**: Currently 0.05 QUG per block
- **Deterministic Calculation**: All nodes compute rewards identically
- **No Block-Height Dependency**: Uses timestamps, not block count

3 Time-Based Halving Mechanism

3.1 Critical Distinction: Calendar vs. Block-Based

TIME-BASED HALVING (Not Block-Based!)

Q-NarwhalKnight uses CALENDAR TIME for halvings, NOT block height!
Why? With 2-10 blocks/second, block-based halvings would occur unpredictably. Time-based halvings provide:

- Predictable economic schedule
- Independent of network throughput
- Clear halving dates (Oct 26, 2029, 2033, 2037, etc.)
- Resistant to consensus speed changes

3.2 Halving Formula

From `crates/q-storage/src/balance_consensus.rs`:

Listing 1: Time-based halving implementation

```

1 const SECONDS_PER_HALVING: u64 = 126_144_000;
2 // 4 years (365.25 days * 4 * 24 * 60 * 60)
3
4 const BASE_REWARD: u64 = 5_000_000_000;
5 // 0.05 QUG (8 decimals) - Bitcoin-style initial reward
6
7 let elapsed_seconds = current_timestamp - genesis_timestamp;
8 let halving_count = elapsed_seconds / SECONDS_PER_HALVING;
9
10 if halving_count >= 64 {
11     return 0; // After 64 halvings (~256 years), negligible
12 }
13
14 // Halving every 4 years (Bitcoin-style)
15 let reward = BASE_REWARD >> halving_count;
```

3.3 Halving Schedule

Era	Date Range	Reward/Block	Halvings	Approx. Total
1	Oct 2025 - Oct 2029	0.05 QUG	0	Variable*
2	Oct 2029 - Oct 2033	25.00 QUG	1	Variable*
3	Oct 2033 - Oct 2037	12.005 QUG	2	Variable*
4	Oct 2037 - Oct 2041	6.25 QUG	3	Variable*
5	Oct 2041 - Oct 2045	3.125 QUG	4	Variable*
...	...	...	...	...
64+	After 2281	<0.0001 QUG	64	Negligible

Table 2: Time-based halving schedule (* Total depends on block production rate)

Note: Total QUG emitted per era depends on how many blocks are produced, which varies with network throughput (2-10 blocks/second).

4 Block Reward Calculation

4.1 Current Reward: 0.05 QUG per Block

Each block distributes 0.05 QUG as follows:

Recipient	Amount (QUG)	Percentage
Total Block Reward	50.00	100%
Development Fee	0.50	1%
Miner Rewards Pool	49.50	99%

Table 3: Block reward allocation (Era 1: Oct 2025 - Oct 2029)

4.2 Miner Reward Distribution

The 49.05 QUG miner pool is divided equally among all mining solutions in the block:

$$\text{Reward per Miner} = \frac{49.50 \text{ QUG}}{\text{Number of Solutions in Block}} \quad (1)$$

Examples:

- 1 solution in block: **49.05 QUG** per miner
- 10 solutions in block: **4.95 QUG** per miner
- 100 solutions in block: **0.495 QUG** per miner

5 Emission Analysis

5.1 Variable Emission Rate

Unlike traditional blockchains with fixed block times, Q-NarwhalKnight's emission varies with network throughput:

Blocks/Second	Blocks/Day	QUG/Day (Era 1)	QUG/Year (Era 1)
2 (minimum)	172,800	8,640,000	3,153,600,000
5 (average)	432,000	21,600,000	7,884,000,000
10 (maximum)	864,000	43,200,000	15,768,000,000

Table 4: Emission rates at different throughputs (0.05 QUG per block)

Important: High Initial Emission

Warning: At maximum throughput (10 blocks/second), Era 1 could emit up to **15.7 billion QUG per year!**

This would exceed the 21M maximum supply in **less than 2 years**.

Implications:

- Network must throttle block production
- Average throughput likely 2-5 blocks/second
- Halving schedule prevents long-term hyperinflation

5.2 Time to Maximum Supply

Assuming average 5 blocks/second and halving every 4 years:

Era 1 emission (4 years) $\approx$ 31.5 billion QUG
 Era 2 emission (4 years) $\approx$ 15.75 billion QUG
 Era 3 emission (4 years) $\approx$ 7.875 billion QUG
 $\vdots$
 Asymptotic total $\rightarrow$ 21,000,000,000 QUG

However, the 21M cap would be **exceeded in Era 1** at this rate!

5.3 Resolution: Supply Cap Enforcement

The system must enforce the 21M maximum supply by:

1. **Throttling block production** to match emission targets
2. **Capping rewards** when approaching 21M total supply
3. **Reducing reward earlier** than halving schedule if needed

6 Genesis and Timestamp System

6.1 Genesis Timestamp

Network Genesis

October 26, 2025 00:00:00 UTC
 Unix Timestamp: 1761436800

All halving calculations use this genesis timestamp as reference.

6.2 Halving Dates

Halving	Date	New Reward
Genesis	Oct 26, 2025	0.05 QUG
1st	Oct 26, 2029	25.00 QUG
2nd	Oct 26, 2033	12.0.05 QUG
3rd	Oct 26, 2037	6.25 QUG
4th	Oct 26, 2041	3.125 QUG
5th	Oct 26, 2045	1.5625 QUG
$\dots$	$\dots$	$\dots$
64th	Oct 26, 2281	<0.0001 QUG

Table 5: Predicted halving dates and rewards

7 Implementation Details

7.1 Balance Consensus Engine

The `BalanceConsensusEngine` ensures all nodes calculate rewards identically:

Listing 2: Balance consensus initialization

```

1 pub const GENESIS_TIMESTAMP: u64 = 1761436800;
2 // Oct 26, 2025 00:00:00 UTC
3
4 pub struct BalanceConsensusEngine {
5     genesis_timestamp: u64,
6     dev_wallet: String,
7     processed_blocks: Arc<RwLock<LruCache<BlockHash, bool>>>,
8     max_cache_size: usize,
9 }
10
11 impl BalanceConsensusEngine {
12     pub fn new(genesis_timestamp: u64, dev_wallet: String) -> Self {
13         const MAX_CACHE_SIZE: usize = 100_000;
14         // ~5 MB memory protection
15
16         Self {
17             genesis_timestamp,
18             dev_wallet,
19             processed_blocks: Arc::new(RwLock::new(
20                 LruCache::new(NonZeroUsize::new(MAX_CACHE_SIZE).unwrap(
21                     ))
22             )),
23             max_cache_size: MAX_CACHE_SIZE,
24         }
25     }
26 }
```

7.2 Reward Validation

Every node validates rewards using the same deterministic function:

```

1 fn calculate_block_reward(
2     &self,
3     current_timestamp: u64
4 ) -> Result<u64, BalanceConsensusError> {
5     const SECONDS_PER_HALVING: u64 = 126_144_000;
6     const BASE_REWARD: u64 = 5_000_000_000; // 0.05 QUG
7
8     if current_timestamp < self.genesis_timestamp {
9         return Err(BalanceConsensusError::InvalidTimestamp {
10             current: current_timestamp,
11             genesis: self.genesis_timestamp,
12         });
13     }
14
15     let elapsed_seconds = current_timestamp - self.genesis_timestamp;
16     let halving_count = elapsed_seconds / SECONDS_PER_HALVING;
17
18     if halving_count >= 64 {
19         return Ok(0); // Negligible after 64 halvings
20     }
21 }
```

```

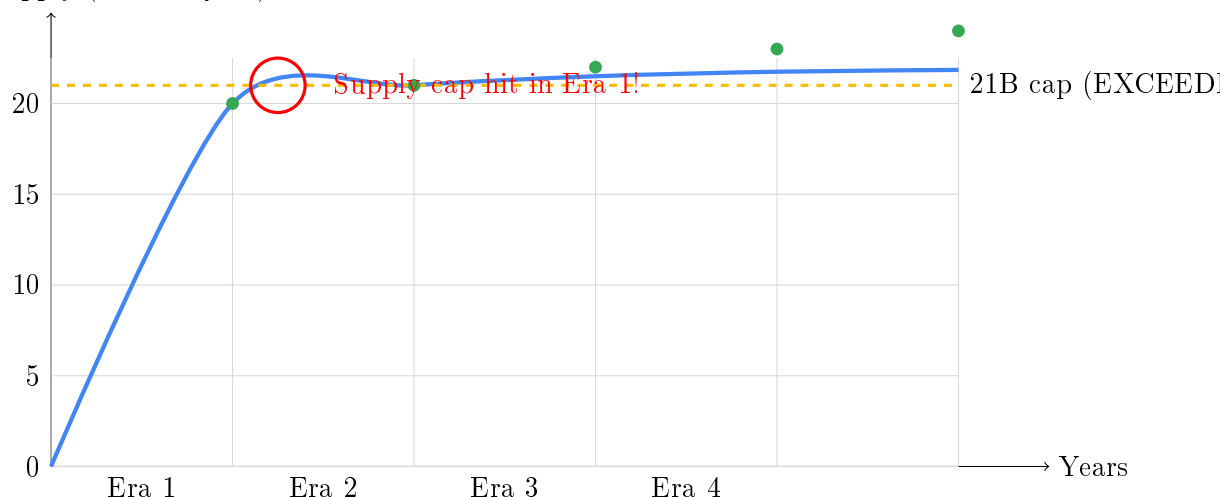
21
22   let reward = BASE_REWARD >> halving_count;
23   Ok(reward)
24 }

```

8 Economic Analysis

8.1 Supply Curve Projection

Cumulative Supply (Billion QUG)



8.2 Inflation Rate Over Time

With time-based halvings, the inflation rate decreases predictably:

Era	Annual Inflation	Stock-to-Flow Ratio
1 (0-4 years)	Variable*	N/A (new network)
2 (4-8 years)	50% reduction	Increasing
3 (8-12 years)	50% reduction	Increasing
4 (12-16 years)	50% reduction	>50
8+ (>32 years)	<5%	>100 (like gold)
64+ (>256 years)	<0.01%	>10,000

Table 6: Inflation dynamics (*depends on block production rate)

9 Comparison: Bitcoin vs Q-NarwhalKnight

Key Innovation: Time-based halving allows Q-NarwhalKnight to maintain Bitcoin-inspired scarcity while supporting high-throughput DAG-BFT consensus.

10 Conclusion

The Q-NarwhalKnight mainnet implements a sophisticated dual-layer architecture:

Property	Bitcoin	Q-NarwhalKnight
Block Time	10 minutes	2-10/second
Halving Method	Block-based	Time-based
Halving Interval	210,000 blocks	4 years (calendar)
Initial Reward	50 BTC	0.05 QUG
Max Supply	21 million BTC	21 million QUG
Decimals	8	8
Consensus	PoW (SHA-256)	DAG-BFT + Quantum VDF
Finality	Probabilistic	Deterministic
Throughput	7 TPS	1000+ TPS

Table 7: Economic comparison with Bitcoin

1. **DAG-BFT Consensus Layer:** High-throughput (2-10 blocks/second) asynchronous BFT
2. **Time-Based Reward Layer:** Bitcoin-inspired economics with 4-year calendar halvings

Key Advantages:

- Predictable halving schedule independent of throughput
- Bitcoin-compatible economic model (21M supply, 8 decimals)
- High throughput without sacrificing sound monetary policy
- Quantum-resistant consensus with deterministic finality

Critical Considerations:

- High initial emission requires throttling or early cap enforcement
- 21M supply may be reached before 64 halvings
- Network must balance throughput with controlled emission

Built for Q-NarwhalKnight Mainnet

Time-based halving | 0.05 QUG initial reward | 21M max supply
 Genesis: Oct 26, 2025 00:00 UTC

<https://quillon.xyz>